

Моделирование предметной области и проектирование БД

План лекции

- 1. От архитектуры к данным: зачем нужно моделирование.
- 2. Методология анализа: от «Как есть» (AS IS) к «Как будет» (TO BE).
- 3. Три уровня моделирования данных: концептуальный, логический, физический.
- 4. Концептуальное моделирование: сущности, атрибуты, связи, кардинальность.
- 5. Логическое проектирование: отображение ER-модели в реляционную схему.
- 6. Нормализация: 1НФ, 2НФ, 3НФ и борьба с аномалиями.
- 7. Денормализация: когда и зачем нарушать правила.
- 8. Физическое проектирование: типы данных, индексы, ограничения.
- 9. Краткий обзор NoSQL подходов (для контекста современных технологий).
- 10. Инструментарий и типичные ошибки проектирования.

Почему проектирование БД критически важно?

- База данных — это фундамент приложения. Ошибки в фундаменте стоят дороже всего.
- Правило 1-10-100:
 - Исправление ошибки на этапе проектирования: 1 единица усилий.
 - Исправление на этапе разработки: 10 единиц усилий.
 - Исправление в работающей системе (продакшен): 100 единиц усилий.
- Плохая модель БД приводит к:
 - Аномалиям обновления (противоречивые данные).
 - Низкой производительности запросов.
 - Сложности масштабирования и рефакторинга.

Методология анализа: выявление потребностей

- Цель: Понять, какие данные нужны бизнесу и как они преобразуются.
- Этапы анализа:
 1. Идентификация бизнес-процессов (что делает система?).
 2. Выявление информационных объектов (о чем храним данные?).
 3. Определение потоков данных (как объекты взаимодействуют?).
 4. Формулировка бизнес-правил и ограничений.
- Arteфакты анализа:
 - Глоссарий предметной области (единая терминология).
 - Список информационных потребностей (какие отчеты и экраны нужны).
 - Черновик ER-диаграммы.

Три уровня моделирования данных

- 1. Концептуальная модель (Что?):
 - – Независима от СУБД и технологий.
 - – Фокус на бизнес-сущностях и их связях.
 - – Аудитория: заказчики, аналитики, архитекторы.
- 2. Логическая модель (Как логически?):
 - – Детализация атрибутов, типов данных (абстрактных), первичных и внешних ключей.
 - – Приведение к нормальным формам (нормализация).
 - – Аудитория: аналитики, разработчики БД.
- 3. Физическая модель (Как физически?):
 - – Конкретная СУБД (PostgreSQL, MySQL).
 - – Реальные типы данных (VARCHAR, INT), индексы, триггеры, партиционирование.
 - – Аудитория: DBA, backend-разработчики.

Концептуальное моделирование: основы ER

- Сущность (Entity): Объект реального мира (Клиент, Заказ).
- Атрибут (Attribute): Свойство сущности (Имя, Дата).
 - Простые vs Составные (Адрес -> Город, Улица).
 - Однозначные (ID) vs Многозначные (Телефоны).
- Связь (Relationship): Ассоциация между сущностями.
- Кардинальность (Мощность связи):
 - 1:1 (Один-к-одному): Пользователь ↔ Паспорт.
 - 1:N (Один-ко-многим): Клиент ↔ Заказы.
 - M:N (Многие-ко-многим): Студент ↔ Курсы.

Отображение ER-модели в реляционную схему

- Правила трансляции:
 1. Каждая сильная сущность становится отдельной таблицей.
 2. Атрибуты становятся колонками таблицы.
 3. Выбирается Первичный ключ (Primary Key, PK).
 4. Обработка связей:
 - – 1:N: Внешний ключ (Foreign Key, FK) добавляется в таблицу на стороне 'N'.
 - – 1:1: FK добавляется в любую из таблиц (или объединяются в одну).
 - – M:N: Создается новая ассоциативная (связующая) таблица, содержащая FK обеих сущностей. Эта пара FK часто становится составным PK.

Нормализация: зачем это нужно?

- Цель: Устранить избыточность данных и аномалии модификации.
- Аномалии:
 - Вставки (Insertion): Нельзя добавить данные об объекте без привязки к другому.
 - Удаления (Deletion): Удаление одной записи ведет к потере других, независимых данных.
 - Модификации (Update): Изменение данных в одном месте требует изменения во многих (риск противоречий).
- Нормальная форма (НФ): Степень соответствия таблицы строгим правилам.
- Золотой стандарт для большинства OLTP-систем: Третья нормальная форма (3НФ).

1НФ, 2НФ, 3НФ: краткий справочник

- Первая нормальная форма (1НФ):
 - – Все атрибуты атомарны (неделимы). Нет списков или JSON в одной ячейке.
 - – Существует первичный ключ.
- Вторая нормальная форма (2НФ):
 - – Выполнена 1НФ.
 - – Каждый неключевой атрибут полностью функционально зависит от ВСЕГО первичного ключа (актуально для составных ключей).
- Третья нормальная форма (3НФ):
 - – Выполнена 2НФ.
 - – Нет транзитивных зависимостей: неключевой атрибут не зависит от другого неключевого атрибута.
 - – Эвристика: «Каждый неключевой атрибут должен сообщать факты о ключе, целом ключе и ни о чем, кроме ключа» (У. Кент).

Денормализация: осознанное нарушение правил

- Что это: Намеренное внесение избыточности в спроектированную БД.
- Зачем:
 - Повышение производительности чтения (избежать дорогих JOIN).
 - Упрощение запросов для аналитики и отчетности (OLAP).
 - Историчность данных (например, сохранить 'Цену на момент заказа' в таблице Заказа, а не ссылаться на актуальную цену Товара).
- Цена денормализации:
 - Усложнение логики обновления данных (нужно обновлять в нескольких местах).
 - Риск рассинхронизации данных.
- Правило: Сначала нормализуйте до 3НФ, денормализуйте только при доказанной необходимости (по результатам профилирования).

Физическое проектирование БД

- Адаптация логической модели под конкретную СУБД (напр., PostgreSQL).
- Ключевые решения:
 1. Выбор точных типов данных (INT vs BIGINT, VARCHAR(n) vs TEXT, DECIMAL для денег).
 2. Ограничения целостности (NOT NULL, UNIQUE, CHECK, DEFAULT).
 3. Проектирование индексов (B-Tree, Hash, GiST) для ускорения частых запросов (WHERE, JOIN, ORDER BY).
 4. Именование объектов (единый стиль: snake_case, префиксы).
 5. Вопросы безопасности: роли, права доступа (GRANT/REVOKE) на уровне таблиц.

Реляционные БД vs NoSQL (контекст ПК-1)

- Реляционные (SQL): PostgreSQL, MySQL. Строгая схема, ACID-транзакции, сложные JOIN. Идеальны для финансовых и учетных систем.
- NoSQL подходы (когда реляционная модель не подходит):
 - – Документориентированные (MongoDB): Гибкая схема, хранение JSON-объектов (каталоги товаров, логи).
 - – Ключ-Значение (Redis): Кэширование, сессии, сверхбыстрое чтение.
 - – Графовые (Neo4j): Социальные сети, рекомендательные системы, поиск путей.
 - – Колоночные (ClickHouse): Аналитика больших данных (OLAP).
- Тренд: Polyglot Persistence (использование разных СУБД для разных задач в одной системе).

Инструменты и управление версиями БД

- Проектирование:
 - – dbdiagram.io, draw.io, ER/Studio, DataGrip (встроенный дизайнер).
- Управление миграциями (Database Version Control):
 - – Проблема: как синхронизировать изменения схемы между разработчиками и средами (Dev -> Test -> Prod)?
 - – Решение: Инструменты миграций (Flyway, Liquibase, Alembic для Python, EF Migrations для .NET).
 - – Принцип: Схема БД хранится в коде (скрипты) и версионизируется в Git вместе с приложением.

Топ-5 ошибок при проектировании БД

- 1. Отсутствие первичных ключей или использование 'бизнес-полей' (например, Email) как РК.
- 2. Игнорирование внешних ключей (Foreign Keys), что ведет к 'висячим' ссылкам.
- 3. Использование универсального типа VARCHAR(255) или TEXT для всего подряд (включая даты и деньги).
- 4. Хранение вычисляемых значений (например, 'Возраст' или 'Итоговая сумма заказа') вместо их вычисления на лету или через триггеры/вьюхи.
- 5. Отсутствие индексов на полях, по которым идет частый поиск или JOIN.

Что должно быть готово в вашем проекте

- К следующему практическому занятию ваш проект должен включать:
- 1. Утвержденную ER-диаграмму (логическая модель, нотация Crow's Foot).
- 2. Обоснование нормализации (все таблицы приведены к 3НФ, либо есть явное обоснование денормализации).
- 3*. Готовый SQL-скрипт создания схемы (DDL):
 - – CREATE TABLE с правильными типами данных.
 - – Ограничения (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE).
 - – Индексы для внешних ключей и часто используемых полей.
- 4*. Скрипт наполнения тестовыми данными (DML: INSERT), достаточными для проверки запросов.

Главные выводы лекции

- 1. Проектирование БД проходит три этапа: концептуальный, логический, физический.
- 2. ER-модель — это язык общения между бизнесом и разработчиками.
- 3. Нормализация (до 3НФ) устраняет аномалии и избыточность.
- 4. Денормализация — это оптимизационный компромисс, а не способ проектирования.
- 5. Физическая модель требует внимания к типам данных, ограничениям и индексам.
- 6. Схема БД должна версионироваться вместе с кодом приложения.

Спасибо за внимание!

- Вопросы?
- Рекомендуемая литература:
 - 1. Дейт К. Дж. «Введение в системы баз данных» (Классика теории).
 - 2. Фаулер М. «Архитектура корпоративных программных приложений» (Главы о работе с БД).
 - 3. Документация PostgreSQL: Глава 5 (Data Definition) и Глава 11 (Indexes).
 - 4. Kleppmann M. «Designing Data-Intensive Applications» (Для понимания компромиссов)